

Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 5 & 6: The Linux and xv6 File System.

Author:
st. gr. FAF-241 Mitrofan Alexia
Verified:
asist. univ. Reițman Patricia

Chișinău - 2026

Contents

OBJECTIVES **2**

LABORATORY 5 **3**

Before the start 3

Part 1. The tree and file types 3

Part 2. Inodes and links 5

Part 3. Permissions 8

Part 4. Disks and the kernel as files 11

LABORATORY 6 **14**

Part 1. Reading what xv6 shows the user 14

Part 2. Writing mystat.c 16

Part 3. Measuring the limits 20

Part 4. Writing find.c 23

CONCLUSION **27**

CONCLUSION **29**

OBJECTIVES

The objectives of laboratory works 5 and 6 are the following:

- Explore the Linux directory tree and identify file types (regular file, directory, symbolic link, character and block device) from the first letter of `ls -l`.
- Understand the difference between an inode and a file name, and observe how hard links, symbolic links and link counts behave when files are removed.
- Distinguish a file's apparent size from the disk space it actually uses (block allocation and sparse files).
- Study the Linux permission model: `rwx` bits and their octal form, the effect of `umask`, the different meaning of `r` and `x` on directories, and the special bits setuid and sticky.
- Inspect the storage layer and the kernel through files: disks, partitions, mount points and file system types (`lsblk`, `df`, `mount`), and the virtual `/proc` file system.

LABORATORY 5

Before the start

In this lab, we have to work as a normal user, not as root, this is why we check beforehand.

```
whoami                    # must not print root
mkdir -p ~/os-lab5 && cd ~/os-lab5
```

```
vboxuser@Ubuntu2604:~$ whoami
vboxuser
vboxuser@Ubuntu2604:~$ mkdir -p ~/os-lab5 && cd ~/os-lab5
vboxuser@Ubuntu2604:~/os-lab5$
```

Checking if we work as normal user and creating working folder

Part 1. The tree and file types

```
ls /
ls -ld /etc /etc/passwd /bin /dev/null /tmp
ls -l /dev | head
file /usr/bin/ls /etc/passwd /bin
du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
```

```
vboxuser@Ubuntu2604:~/os-lab5$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var
vboxuser@Ubuntu2604:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 16 22:03 /dev/null
drwxr-xr-x 142 root root 12288 Oct 13 11:18 /etc
-rw-r--r--   1 root root  2823 Oct  1 10:55 /etc/passwd
drwxrwxrwt  15 root root   360 Oct 16 22:04 /tmp
vboxuser@Ubuntu2604:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--  1 root     root     10, 235 Oct 16 22:03 autofs
drwxr-xr-x  2 root     root         500 Oct 16 22:03 block
drwxr-xr-x  2 root     root          80 Oct 16 22:03 bsg
crw-------  1 root     root     10, 234 Oct 16 22:03 btrfs-control
drwxr-xr-x  3 root     root          60 Oct 16 22:03 bus
lrwxrwxrwx+ 1 root     root           3 Oct 16 22:03 cdrom -> sr0
drwxr-xr-x  2 root     root        3780 Oct 16 22:03 char
crw-------  1 root     tty        5,   1 Oct 16 22:03 console
lrwxrwxrwx  1 root     root          11 Oct 16 22:03 core -> /proc/kcore
vboxuser@Ubuntu2604:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
vboxuser@Ubuntu2604:~/os-lab5$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
141M    /var/log
5.5G    /usr
vboxuser@Ubuntu2604:~/os-lab5$
```

Figure 1.1: Checking the tree and file types

1. **For each path in the second line, write its type from the first letter of `ls -l`.**
   The types are the following:
   l (symbolic link file), seen in `/bin -> usr/bin`;
   c (character device file), seen in `/dev/null`;
   d (directory), seen in `/etc` and `/tmp`;
   - (regular file), seen in `/etc/passwd`.

2. **What is `/bin` on your system: a directory or a link, and to where?**
   In my system, `/bin` is a symbolic link file, and it points to `/usr/bin`. More precisely, in the screenshot it's a relative link in this case, and it points to `/usr/bin` relative to `/`.

3. **Find one character device and one block device in `/dev`.**
   From the figure 1.1, we can see that `autofs`, `btrfs-control` and `console` are character devices. In the screenshot there are no block devices, but we can see them with the following command: `ls -l /dev/block`, and block devices are `sr0`, `loop` (from 0 to 18) and `sda` (from 0 to 2).

4. **Which is bigger on your laptop, `/var/log` or `/usr`, and by roughly how much?**
   `/usr` is bigger than `/var/log` by roughly 40 times. `/usr` is 5.5G, and `/var/log` is 141M.

Part 2. Inodes and links

```
echo hello > notes.txt
stat notes.txt
ls -li notes.txt
```

```
vboxuser@Ubuntu2604:~/os-lab5$ echo hello > notes.txt
vboxuser@Ubuntu2604:~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 307339      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/vboxuser)   Gid: ( 1000/vboxuser)
Access: 2026-10-16 22:07:57.431141171 +0000
Modify: 2026-10-16 22:07:57.364074283 +0000
Change: 2026-10-16 22:07:57.364074283 +0000
 Birth: 2026-10-16 22:07:57.364074283 +0000
vboxuser@Ubuntu2604:~/os-lab5$ ls -li notes.txt
307339 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 16 22:07 notes.txt
vboxuser@Ubuntu2604:~/os-lab5$
```

Figure 2.1: An inode, and a name that is not part of it

```
ln notes.txt hard.txt
ls -li notes.txt hard.txt
rm notes.txt
cat hard.txt ; ls -li hard.txt
```

```
vboxuser@Ubuntu2604:~/os-lab5$ ln notes.txt hard.txt
vboxuser@Ubuntu2604:~/os-lab5$ ls -li notes.txt hard.txt
307339 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 16 22:07 hard.txt
307339 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 16 22:07 notes.txt
vboxuser@Ubuntu2604:~/os-lab5$ rm notes.txt
vboxuser@Ubuntu2604:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
307339 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 16 22:07 hard.txt
```

Figure 2.2: Hard link, then removing the "original" file

```
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt
cat soft.txt ; ls -l soft.txt
```

```
vboxuser@Ubuntu2604:~/os-lab5$ ln -s hard.txt soft.txt
vboxuser@Ubuntu2604:~/os-lab5$ ls -li hard.txt soft.txt
307339 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 16 22:07 hard.txt
271041 lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 16 22:09 soft.txt -> hard.txt
vboxuser@Ubuntu2604:~/os-lab5$ cat soft.txt
hello
vboxuser@Ubuntu2604:~/os-lab5$ rm hard.txt
vboxuser@Ubuntu2604:~/os-lab5$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 16 22:09 soft.txt -> hard.txt
```

Figure 2.3: Symbolic link, then removing its target

```
1 mkdir d ; ls -ld d
2 mkdir d/x d/y ; ls -ld d
```

```
vboxuser@Ubuntu2604:~/os-lab5$ mkdir d ; ls -ld d
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 16 22:10 d
vboxuser@Ubuntu2604:~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 vboxuser vboxuser 4096 Oct 16 22:10 d
vboxuser@Ubuntu2604:~/os-lab5$
```

Figure 2.4: Directories and their links

```
1 echo x > one.txt ; ls -l one.txt ; du -h one.txt
2 truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
3 rm big.img
```

```
vboxuser@Ubuntu2604:~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 vboxuser vboxuser 2 Oct 16 22:10 one.txt
4.0K    one.txt
vboxuser@Ubuntu2604:~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 vboxuser vboxuser 1.0G Oct 16 22:11 big.img
0       big.img
vboxuser@Ubuntu2604:~/os-lab5$ rm big.img
```

Figure 2.5: Symbolic link, then removing its target

1. **Which number in stat is the inode, and which tells how many names the file has?**
   The file's inode is 307339. We can see it in both `stat notes.txt`, in `Inode:` column, and `ls -li notes.txt`, as the first number in the line. We can see the number of names in the `Links:` column, and in our case it is `1`.

2. **After `rm notes.txt`, why could you still read `hard.txt`? What does rm really remove?**

We can still read `hard.txt` because in the command `ln notes.txt hard.txt`, we linked those two files. To be more precise, we created a second name for the same file, both pointing to inode 307339. A file is actually its inode (data and metadata), and a filename is just a directory entry that points to an inode. `rm` doesn't actually delete the file, it removes the name (directory entry) we created, and by doing this, we decrement the link count by 1. Here the count went from 2 to 1, so `hard.txt` still printed `hello`. The data is freed only when the link count reaches 0 and no process still has the file open.

3. **Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?**
   This time we had a symbolic link, which is similar to a shortcut: `soft.txt`, by being a symbolic link, is actually a separate file with its own inode (271041, we can see it in figure 2.3). It refers to a name, and its content is the path text it points to. After `rm hard.txt`, that name no longer exists, so the link is dangling (broken) and cat fails.
   The size is 8 bytes because the link stores the string **hard.txt**, which is 8 characters (`-a-r-d-.-t-x-t`).

4. **A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.**
   Link 1: the entry `d` in its parent directory (`~/os-lab5`);
   Link 2: the entry . inside d itself;
   Links 3 and 4: the .. entries inside `d/x` and `d/y`, which point back to `d`.
   So we can use this this formula to determine link count: link count = 2 + number of subdirectories.

5. **Why does a 2-byte file use 4.0K, and a 1 GB file use 0 ?**
   For `one.txt`, the filesystem allocates space in whole blocks, so 4096 bytes (4 KiB). Even 2 bytes (`x` and `/n`)take one block, so `du` shows 4.0K.
   For `big.img`, `truncate -s 1G` only sets the file size, it doesn't allocate data blocks. So the unwritten regions or "holes" read back as zeros but don't take any disk space. Thus `ls` reports 1.0G and `du` reports 0. That is why `rm big.img` was instant and freed nothing.

Part 3. Permissions

```
echo secret > s.txt
chmod 600 s.txt ; ls -l s.txt
chmod 000 s.txt ; cat s.txt
chmod u+r s.txt ; cat s.txt
umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
```

```
vboxuser@Ubuntu2604:~/os-lab5$ echo secret > s.txt
vboxuser@Ubuntu2604:~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 vboxuser vboxuser 7 Oct 16 22:11 s.txt
vboxuser@Ubuntu2604:~/os-lab5$ chmod 600 s.txt ; cat s.txt
secret
vboxuser@Ubuntu2604:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
vboxuser@Ubuntu2604:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 vboxuser vboxuser    0 Oct 16 22:15 new.txt
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 16 22:15 newdir
```

Figure 3.1: Working with files

```
mkdir box ; echo hi > box/f
chmod u=r,go= box  ; ls box ; cat box/f      # read only
chmod u=x,go= box  ; ls box ; cat box/f      # execute only
chmod 755 box
```

```
vboxuser@Ubuntu2604:~/os-lab5$ mkdir box ; echo hi > box/f
vboxuser@Ubuntu2604:~/os-lab5$ chmod u=r,go= box  ; ls box ; cat box/f
f
cat: box/f: Permission denied
vboxuser@Ubuntu2604:~/os-lab5$ chmod u=x,go= box  ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
vboxuser@Ubuntu2604:~/os-lab5$ chmod 755 box
vboxuser@Ubuntu2604:~/os-lab5$
```

Figure 3.2: Working with directories

```
ls -l /usr/bin/passwd ; ls -ld /tmp
cat /etc/shadow
ls -l /etc/shadow
```

```
vboxuser@Ubuntu2604:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 15 root root 360 Oct 16 22:15 /tmp
vboxuser@Ubuntu2604:~/os-lab5$ cat /etc/shadow
cat: /etc/shadow: Permission denied
vboxuser@Ubuntu2604:~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1336 Oct  1 10:56 /etc/shadow
vboxuser@Ubuntu2604:~/os-lab5$
```

Figure 3.3: Special bits and other people's files

1. **Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.**

   600: `rw-------`; owner can read and write, no one else has access. Common for SSH keys and sensitive data;

   644: `rw-r--r--`; owner can read and write, group and others can only read. Common for documents and configuration files;

   755: `rwxr-xr-x`; owner can do anything, group and others can read and execute. Used for scripts, programs and directories (anyone can enter and list it);

   700: `rwx------`; owner can do anything, no one else has access. Used for private directories, programs, what SSH requires for `~/ssh`.

2. **With `chmod 000` you own the file but cannot read it. Why does ownership not help?**

   Ownership doesn't help because it doesn't grant access by itself. The kernel, checking 000, sees that the owner bits are empty, so we are denied, and we can't override it. As a owner, however, we can change permissions, so we can use that to access files. Only root bypasses permission bits entirely.

3. **What is your `umask`, and how does it explain the permissions of `new.txt` and `newdir`?**

   My umask, as of figure 3.1, is 0002. The leading 0 is for the special permissions (setuid, setgid, and sticky, and it is assumed to be there. The following 0 is for owner, and it means that any permission may be set (read, write, execute). The third 0 is for group, and like owner, any permissions may be set. The last 2 is for others, and it means that setting of write permission is prohibited (only read and execute).

   Technically, with `umask`, we remove permissions from defaults: programs request 666 for files and 777 for directories, and umask is substracted bitwise. 666 minus 002 is 664, and its permissions in rwx format are `rw-rw-r--`, which matches `new.txt`, and 777 minus 002 is 775, or `rwxrwxr-x` in rwx format, which matches `newdir`. This can all be seen in figure 3.1.

4. **Explain the box experiment: what does `r` allow on a directory, and what does `x` allow?**
   If we can read a directory, we can list its contents (`ls`), and if we can "execute" a directory, we can move through its hierarchy, even if we don't know what is inside (`cd`).

5. **What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does passwd need the `s`?**
   In the first case, the `s` replaces the owner's x: `-rwsr-xr-x`, and it is the setuid bit. When a user runs the progra, it executes with the privileges of the file's owner (root), not the user running it. `passwd` needs it because passwords are stored in `/etc/shadow`, which we cannot access, however a normal user must be able to change their own password, which means writing to that file. Setuid lets `passwd` run as root for that one purpose, so it can update only the user's own entry, while users still cannot touch the file directly.
   In the second case, `t` is the sticky bit, and it replaces the other's execute: `drwxrwxrwt`. In `/tmp`, everyone can create files, but only a file's owner (or the directory's owner, or root) can delete or rename it. Without it, anyone with write permission on `/tmp` could delete other users' files.

Part 4. Disks and the kernel as files

```
lsblk
df -h
mount | grep " / "
grep -c processor /proc/cpuinfo
head -3 /proc/meminfo
head -5 /proc/self/status
ls -l /proc/self/fd
```

```
vboxuser@Ubuntu2604:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/1643
loop2    7:2    0  66.8M  1 loop /snap/core24/2124
loop3    7:3    0    20M  1 loop /snap/desktop-security-center/151
loop4    7:4    0  14.2M  1 loop /snap/desktop-security-center/206
loop5    7:5    0 260.3M  1 loop /snap/firefox/8763
loop6    7:6    0  16.5M  1 loop /snap/firmware-updater/226
loop7    7:7    0 614.5M  1 loop /snap/gnome-46-2404/164
loop8    7:8    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop9    7:9    0   1.5M  1 loop /snap/hwctl/123
loop10   7:10   0   1.5M  1 loop /snap/hwctl/131
loop11   7:11   0   402M  1 loop /snap/mesa-2404/1839
loop12   7:12   0  18.8M  1 loop /snap/prompting-client/222
loop13   7:13   0  18.4M  1 loop /snap/prompting-client/228
loop14   7:14   0  50.1M  1 loop /snap/snapd/27710
loop15   7:15   0  11.8M  1 loop /snap/snap-store/1390
loop16   7:16   0  44.7M  1 loop /snap/snapd/28254
loop17   7:17   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0    25G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0    25G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/vboxuser/VBox_GAs_7.2.20
```

Figure 4.1: Printing all block devices

```
vboxuser@Ubuntu2604:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.5G  2.1M  1.5G   1% /run
/dev/sda2        25G  8.3G   15G  36% /
tmpfs           3.6G     0  3.6G   0% /dev/shm
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           3.6G  8.0K  3.6G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           724M   88K  724M   1% /run/user/1000
/dev/sr0         51M   51M     0 100% /run/media/vboxuser/VBox_GAs_7.2.20
vboxuser@Ubuntu2604:~/os-lab5$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
vboxuser@Ubuntu2604:~/os-lab5$ grep -c processor /proc/cpuinfo
4
vboxuser@Ubuntu2604:~/os-lab5$ head -3 /proc/meminfo
MemTotal:        7407688 kB
MemFree:         3894096 kB
MemAvailable:    5657976 kB
vboxuser@Ubuntu2604:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   5106
Ngid:   0
vboxuser@Ubuntu2604:~/os-lab5$ ls -l /proc/self/fd
total 0
lrwx------ 1 vboxuser vboxuser 64 Oct 16 22:19 0 -> /dev/pts/0
lrwx------ 1 vboxuser vboxuser 64 Oct 16 22:19 1 -> /dev/pts/0
lrwx------ 1 vboxuser vboxuser 64 Oct 16 22:19 2 -> /dev/pts/0
lr-x------ 1 vboxuser vboxuser 64 Oct 16 22:19 3 -> /proc/5109/fd
```

Figure 4.2: Checking disk filesystem

1. **Draw your disk: disk name, partitions, which one is mounted at / and with what file system type.**

```
sda (25G virtual disk)
├─ sda1   1M    (no mount point)
└─ sda2   25G   mounted at /   (ext4, rw, relatime)
```

   The disk is sda (25G), with two partitions: sda1 (1M) and sda2 (25G). sda2 is mounted at / and its file system type is ext4. The disk is `sda` (25G), and it has two partitions: `sda1` (1M) and `sda2` (25G). `/dev/sda2` is mounted on `/` with type ext4. We can see this from the command `mount | grep " / "`. Ext4 is the default on the Ubuntu.

2. **How full is your root file system?**
   My root system is 8.3G full out of 25G available, with 36% of it used and 15G remaining. It can be seen in the first command in figure 4.2, in the `/dev/sda2` row.

3. **How many CPU cores and how much RAM does your laptop have, ac-**

cording to `/proc`?

According to proc, I have 4 cores, and 7.4GB (7407688 kB) RAM. Those can be seen under the commands `grep -c processor /proc/cpuinfo` and `head -3 /proc/meminfo`, respectively.

4. **`/proc/self/status` showed the name of a program. Which one, and why that one?**

   It shows the program `head`: we ran `head -5 /proc/self/status`, and `/proc/self` is a special link that points to the process that is reading it, which in our case is `head`.

LABORATORY 6

Part 1. Reading what xv6 shows the user

```
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 35376
...
console         3 25 0
$ mkdir a
$ ls a
```

```
init: starting sh
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 36728
echo            2 4 35592
sleep           2 5 35280
args            2 6 35272
forktest        2 7 18136
grep            2 8 44184
init            2 9 35976
kill            2 10 35512
ln              2 11 35304
ls              2 12 42880
mkdir           2 13 35568
rm              2 14 35552
sh              2 15 58312
stressfs        2 16 36416
usertests       2 17 206800
grind           2 18 51824
wc              2 19 37728
zombie          2 20 34848
logstress       2 21 37560
forphan         2 22 36352
dorphan         2 23 35800
sync            2 24 34944
console         3 25 0
$ mkdir a
$ ls a
.               1 26 32
..              1 1 1024
$
```

Figure 5.1: All contents inside root and directory a

1. **What do types 1, 2 and 3 mean (`kernel/stat.h`)?**
   1 means directory, **2** means regular file, **3** means device.

2. **Which inode number does the `root` directory have, and why are `.` and `..` the same there?**
   `root` has the inode number 1, it is the third column after we input `ls` command. `.` is the same, because `.` points to itself, and `..` points to the parent, and since `root` has no parent directory, `..` also points to itself.

3. **An empty directory `a` has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does `a` contain?**
   `a` contains 2 entries: `.`, which is `a` itself, and `..`, which is its parent - `root`. Since these entries have the size of 32 bytes, this means that one entry has the size of 16 bytes (2 bytes for inode, 14 for the name).

Part 2. Writing mystat.c

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  // TODO: return "directory", "file" or "device" for T_DIR, T_FILE, T_DEVICE
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    // TODO: call stat(argv[i], &st); on error print to fd 2 and continue
    // TODO: print  NAME: TYPE, inode N, links N, size N bytes
  }
  exit(0);
}
```

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t) {
     case T_DIR:
        return "directory";
     case T_FILE:
        return "file";
     case T_DEVICE:
        return "device";
     default:
        return "?";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    if (stat(argv[i], &st) < 0) {
        fprintf(2, "stat: cannot stat %s\n", argv[i]);
        continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
        argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Figure 6.1: mystat.c completed

```
132 UPROGS=\
133         $U/_cat\
134         $U/_echo\
135         $U/_sleep\
136         $U/_args\
137         $U/_mystat\
138         $U/_forktest\
139         $U/_grep\
140         $U/_init\
141         $U/_kill\
```

Figure 6.2: mystat.c inside UPROGS

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
$ rm README
$ mystat r2
$ cat r2 | wc
```

```
xv6 kernel is booting

hart 1 starting
init: starting sh
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$
$ cat r2 | wc
48 336 2441
$
```

Figure 6.3: Using mystat.c to observe hard links

1. **Paste the outputs after `ln` and after `rm`. What changed and what did not?**

   After `ln`:

   `README: file, inode 2, links 2, size 2441 bytes`

   `r2: file, inode 2, links 2, size 2441 bytes;`

   After `rm`:

   `r2: file, inode 2, links 1, size 2441 bytes.`

   Here we made a hard link between `README` and `r2`, and after `rm README` we removed `README`, however since these 2 files had been linked we can still access the

contents of `README` through `r2`. This is because two files that are hard linked are basically just two different names for the same inode.

2. **Find the line in `kernel/sysfile.c` (`sys_unlink`) that decrements the link count. When are the data blocks freed?**
   The line that decrements the link count is the following: `ip->nlink-;`. Data blocks are freed when the link count is 0 and no process has the file open.

3. **xv6 has no symbolic links and no permissions. Look at `struct dinode` in `kernel/fs.h`: which fields does an xv6 inode have?**
   An xv6 inode has the following fields:

```
struct dinode {
  short type;               # File type (file, directory or device)
  short major;              # Major device number (T_DEVICE only)
  short minor;              # Minor device number (T_DEVICE only)
  short nlink;              # Number of links(directory entries) to
     this inode in file system
  uint size;                # Size of file (bytes)
  uint addrs[NDIRECT + 1]; # Data block addresses
};
```

   What is missing, compared to Linux: there is no owner (uid/gid), no permission bits, no timestamps, and no file name. The name lives only in directory entries. And there is no symlink type, so the "link" in `ln` is always a hard link, which is why `ln` is enough to see the link count work.

Part 3. Measuring the limits

```
grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ"
    kernel/fs.h
# while building, make prints the disk layout (look for the nmeta line)
make clean ; make fs.img 2>&1 | grep nmeta
$ mkdir averyveryverylongname
$ ls
```

```
vboxuser@Ubuntu2604:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define
DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
vboxuser@Ubuntu2604:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_sleep user/_args user/_mystat user/_forktest user/_grep user/_init use
r/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grin
d user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
vboxuser@Ubuntu2604:~/xv6-riscv$
```

Figure 7.1: Viewing pre-defined block sizes and the total of blocks on the disk

```
$ mkdir averyveryverylongname
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 36728
echo            2 4 35592
sleep           2 5 35280
args            2 6 35272
mystat          2 7 36232
forktest        2 8 18136
grep            2 9 44184
init            2 10 35976
kill            2 11 35512
ln              2 12 35304
ls              2 13 42880
mkdir           2 14 35568
rm              2 15 35552
sh              2 16 58312
stressfs        2 17 36416
usertests       2 18 206800
grind           2 19 51824
wc              2 20 37728
zombie          2 21 34848
logstress       2 22 37560
forphan         2 23 36352
dorphan         2 24 35800
sync            2 25 34944
console         3 26 0
averyveryveryl 1 27 32
$
```

Figure 7.2: Viewing the directory with a long name in xv6's ls

1. **What does `ls` show for `averyveryverylongname`, and which constant explains it?**
   `ls` doesn't show the name fully, only `averyveryveryl`. We can see in figure 7.1 that the directory entry size is predefined: `#define DIRSIZE 14`, which means that if we create a directory with a name longer than 14 bytes, it gets truncated to fit this limit.

2. **Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of `BSIZE` bytes. Show the calculation.**
   The formula is the following:
   `(NDIRECT + NINDIRECT/4) × BSIZE`
   and if we replace with the numbers that are defined in the header we get:
   $\mathbf{(12 + 1024/4) \times 1024 = 274432}$
   So the maximum size a xv6 file can hold is 274432 bytes, or about 268 KiB.

3. **From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?**
   The whole disk has 2000 total blocks: 1 for boot, 1 for super, 31 for log, 13 for inode, 1 for bitmap locks, and 1953 for the data.

Part 4. Writing find.c

Open *find.c ~/xv6-riscv/user Save

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, 0)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }
  if(st.type != T_DIR){        // a plain file has nothing to search inside
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';                  // buf is now "path/", p points just after the slash

```

Figure 8.1: find.c (part 1)

```
33   strcpy(buf, path);
34   p = buf + strlen(buf);
35   *p++ = '/';                  // buf is now "path/", p points just after the slash
36
37   while(read(fd, &de, sizeof(de)) == sizeof(de)){
38     if(de.inum == 0)
39       continue;
40     if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
41       continue;
42
43     memmove(p, de.name, DIRSIZ);
44     p[DIRSIZ] = 0;             // buf is now the full path, properly terminated
45
46     if(strcmp(p, name) == 0)   // last component matches
47       printf("%s\n", buf);
48
49     if(stat(buf, &st) < 0){
50       fprintf(2, "find: cannot stat %s\n", buf);
51       continue;
52     }
53     if(st.type == T_DIR)
54       find(buf, name);         // recurse
55   }
56   close(fd);
57 }
58
59 int
60 main(int argc, char *argv[])
61 {
62   if(argc != 3){
63     fprintf(2, "usage: find <dir> <name>\n");
64     exit(1);
65   }
66   find(argv[1], argv[2]);
67   exit(0);
```

Figure 8.2: find.c (part2)

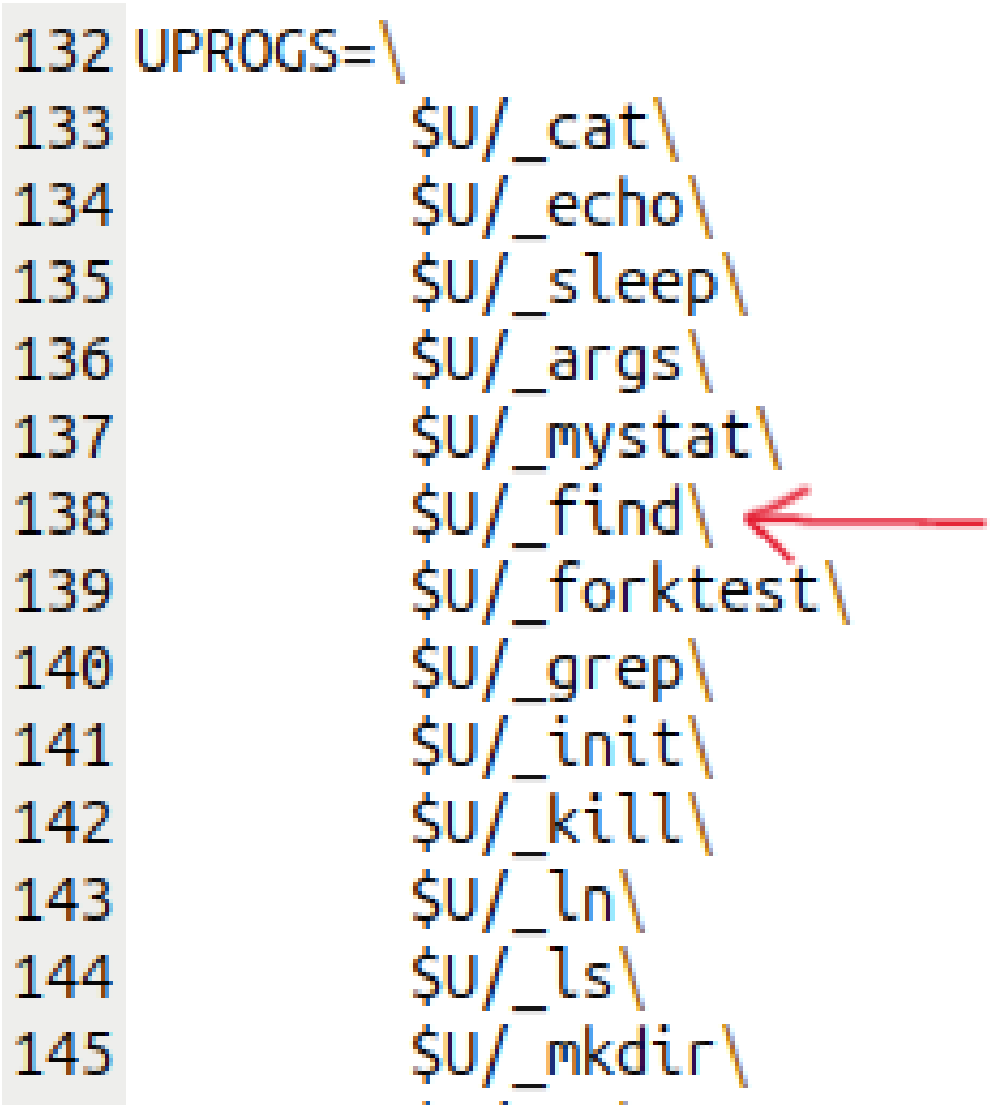


Figure 8.3: find.c inside UPROGS

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

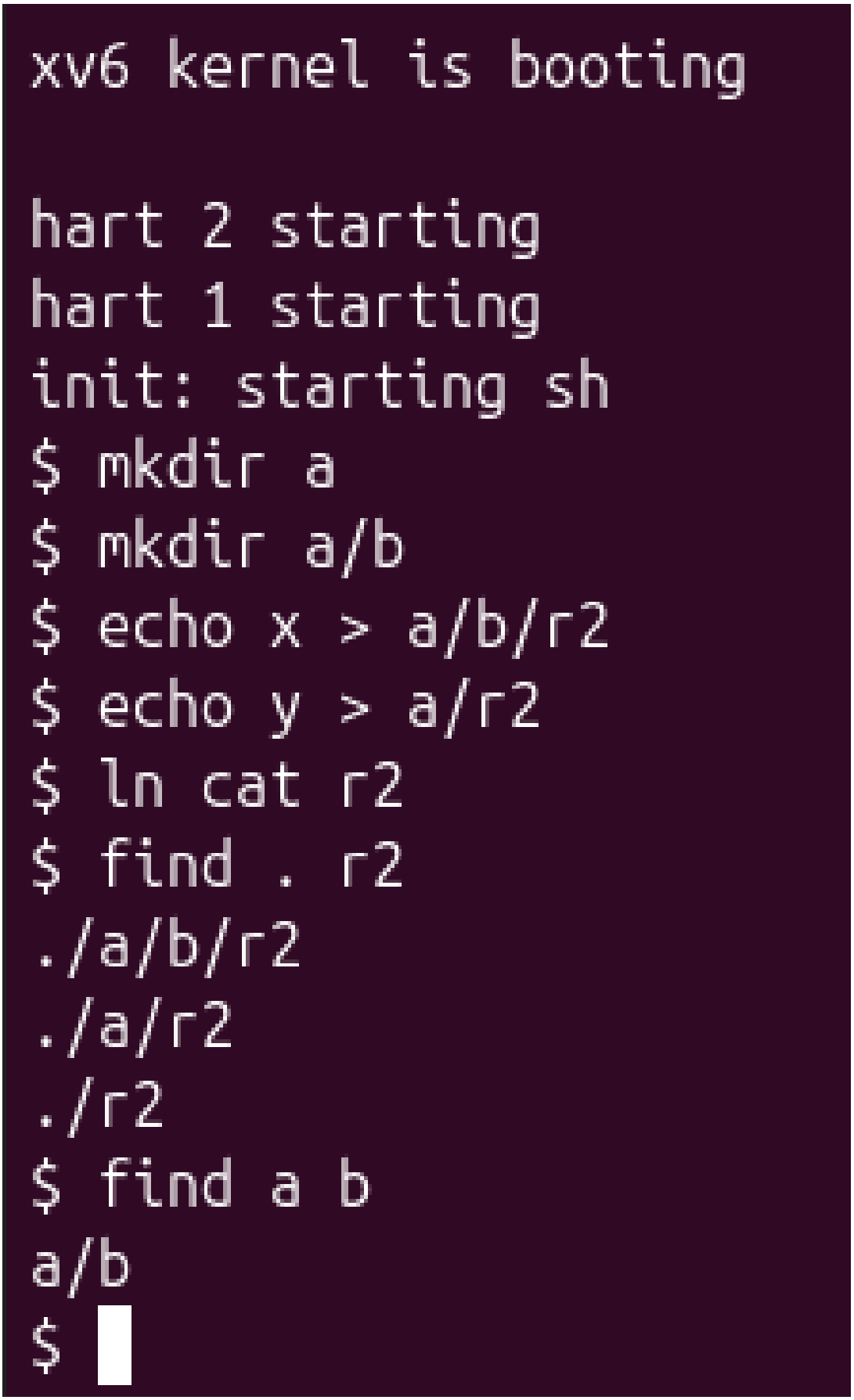


Figure 8.4: Results of find.c

CONCLUSION

In these two labs I studied the same file system design from two sides: Linux with ext4 as a complete, real-world system, and xv6 as a minimal system whose source code I could read, measure and extend. The central idea in both is that a file name is not the file. A name is only an entry in a directory that points to an inode, and the inode holds the metadata and the addresses of the data blocks. This explained the main results of Lab 5: after `rm notes.txt`, `hard.txt` still printed `hello`, because `rm` removes one name and decreases the link count, and the data is freed only when the count reaches zero and no process has the file open. A symbolic link behaves differently: it has its own inode and stores only a path (8 bytes for `hard.txt`), so it became dangling after its target was removed. The link counts of directories follow the same logic: a new directory has 2 links (its name in the parent and its own `.`), and each subdirectory adds one through its `..`. I also learned that size is not disk usage: a 2-byte file occupies a whole 4 KiB block, while a 1 GiB sparse file created with `truncate` uses no blocks at all.

The permission experiments showed that Linux checks only one class (owner, group or others), so owning a file with mode `000` does not allow reading it, although the owner can change the mode. My `umask` is `0002`, which is why new files get `664` and new directories `775`. On directories, `r` allows listing the names and `x` allows reaching entries by name. The setuid bit lets `passwd` update `/etc/shadow` with root privileges, and the sticky bit on `/tmp` protects users' files from deletion by others. Looking at the disk and `/proc` showed that the kernel exposes its state through files: the system uses `/dev/sda2` (ext4, 25 GiB disk, 36% used), has 4 CPUs and about 7 GiB of RAM, and `/proc/self` refers to the process that reads it.

In Lab 6 the same concepts appeared in a much smaller form. xv6 directories are ordinary files made of 16-byte entries (a 2-byte inode number and a 14-byte name), with the root directory at inode 1, whose `..` points to itself. My `mystat` program confirmed hard-link behaviour: after `ln` the file had link count 2, after `rm README` only the count changed to 1, and `cat r2 | wc` still returned `48 336 2441`. In the kernel, `ip->nlink--` in `sys_unlink` lowers the count, and `iput` frees the blocks when it reaches zero. The constants in `kernel/fs.h` explained the limits I measured: `DIRSIZ = 14` truncated the long directory name to `averyveryveryl`, the maximum file size is $(12 + 1024/4) \times 1024 = 274{,}432$ bytes, and the `nmeta` line showed 47 metadata blocks out of 2000, leaving 1953 data blocks. The xv6 inode has no owner, permissions, timestamps or symbolic links, which shows what Linux adds on top of the basic mechanism.

Writing `find` connected these ideas in one program. It opens a directory, reads `struct dirent` entries one by one, builds the full path in a buffer (`path/` followed by the name, terminated with `0`), compares the last component with `strcmp`, and calls `stat` on

each entry to decide whether to recurse into a directory. Entries with `inum == 0` are free slots and must be skipped, and `.` with `..` must be skipped to avoid infinite recursion. I also added checks for a path longer than the 512-byte buffer and for files that cannot be opened or examined, and I use `fstat` to return immediately when the starting path is a plain file. One limit I noticed is that each recursion level keeps its directory open until its loop ends, so the depth is bounded by the 16 open files per process allowed in xv6, and the 512-byte buffer on each level also uses the small user stack. This is acceptable for the test trees but would need changes for very deep ones.

Overall, the labs showed me that the structures behind everyday commands (inodes, directory entries, link counts and block allocation) are simple, and that observing them with real tools and then reading or writing the code made the lecture concepts concrete.

REFERENCES

1. https://github.com/mit-pdos
2. https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf
3. https://www.redhat.com/en/blog/inodes-linux-filesystem
4. https://blog.marcnuri.com/linux-file-permissions-complete-guide
5. https://www.unitconverters.net/data-storage/gb-to-mb.htm
6. https://unix.stackexchange.com/questions/10883/where-do-files-go-when-the-rm-command-is-issued
7. https://www.maths.cam.ac.uk/computing/linux/unixinfo/perms
8. https://www.reddit.com/r/linuxmint/comments/y0bdbt/umask_0002_what_does_it_mean/
9. https://en.wikipedia.org/wiki/Umask#Shell_command
10. https://stackoverflow.com/questions/15355958/what-does-the-x-mean-in-rwx-on-a-directory
11. https://unix.stackexchange.com/questions/21251/execute-vs-read-bit-how-do-directory-permissions-in-linux-work#21252
12. https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/proc.html
13. https://man7.org/linux/man-pages/man8/lsblk.8.html
14. https://man7.org/linux/man-pages/man1/ln.1.html